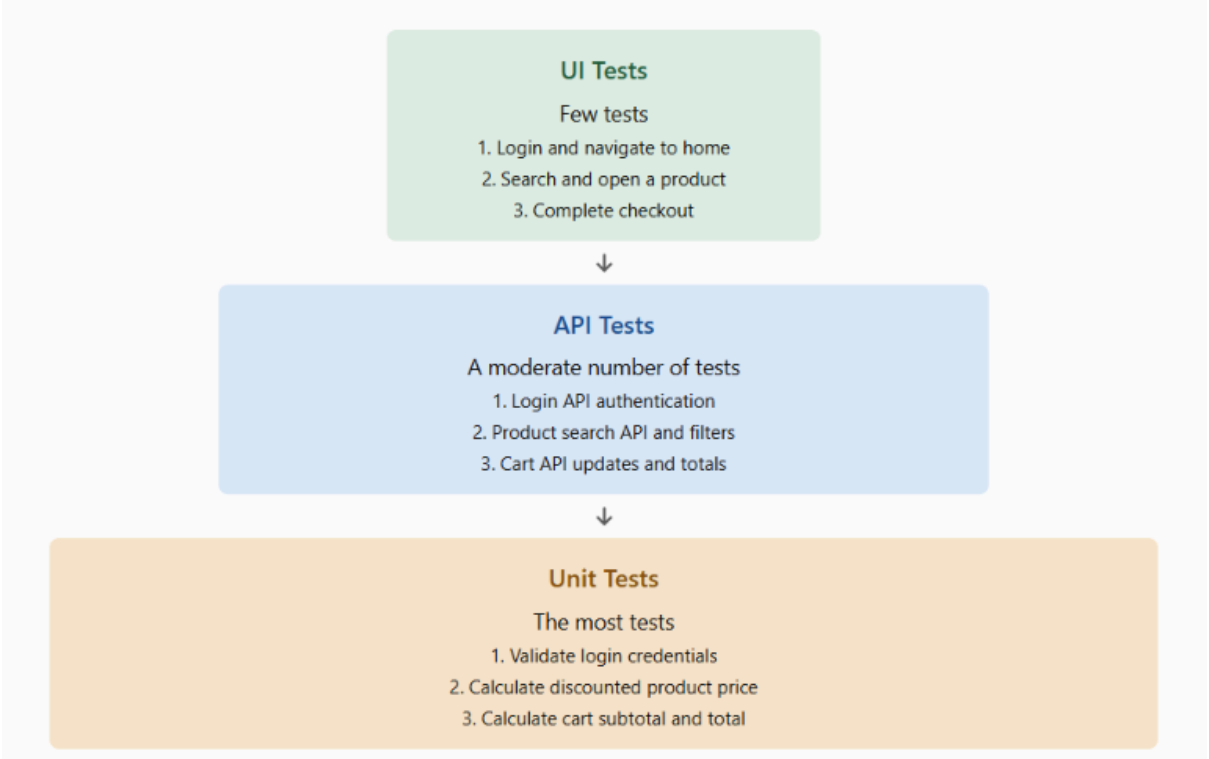
1. Test Scenarios and Classification

#	Test scenario	Classification	Justification
1	Verify that the login function accepts valid credentials and rejects invalid ones.	Unit	Tests the login validation logic independently of the user interface or external services.
2	Verify that a product's final price is calculated correctly after applying a discount.	Unit	Checks the calculation logic using different prices and discount values.
3	Verify that adding an item to the cart updates the quantity and total price correctly.	Unit	Tests the cart calculation and quantity-management logic in isolation.
4	Verify that the login API returns a valid authentication token for correct credentials and an appropriate error for incorrect credentials.	API	Tests the backend authentication endpoint, response status, and response body directly.
5	Verify that the product search API returns products matching the search keyword and filters.	API	Checks backend search results, filtering behavior, and response data without using the UI.
6	Verify that the cart API correctly adds or removes products and returns the updated cart total.	API	Validates cart endpoints, server-side calculations, and data consistency.
7	Verify that a user can log in through the login page and navigate to the home page successfully.	UI	Tests the visible login form, user interactions, and navigation in the application.

8	Verify that a user can search for a product, view the results, and open a product details page.	UI	Tests the search field, displayed results, and navigation between screens.
9	Verify that a user can complete checkout by entering valid delivery details and confirming the order.	UI	Tests the end-to-end interaction with checkout forms, validation messages, and order confirmation screens.
10	Verify that a payment request with valid details completes successfully and returns the correct payment status.	API	Tests the payment endpoint and its response, including successful or failed payment outcomes.

2. Test Pyramid Diagram



Which Layer Has the Most Tests and Why?

Unit Testing should have the most tests, because unit tests:

- Execute quickly and provide fast feedback.
- Test individual functions and business rules in isolation.

- Allow many input combinations and edge cases to be tested efficiently.